How I turned Andrej Karpathy’s LLM Wiki into a tool that writes wiki’s from code

12 min read · 5 days ago

Balu Kosuri Follow

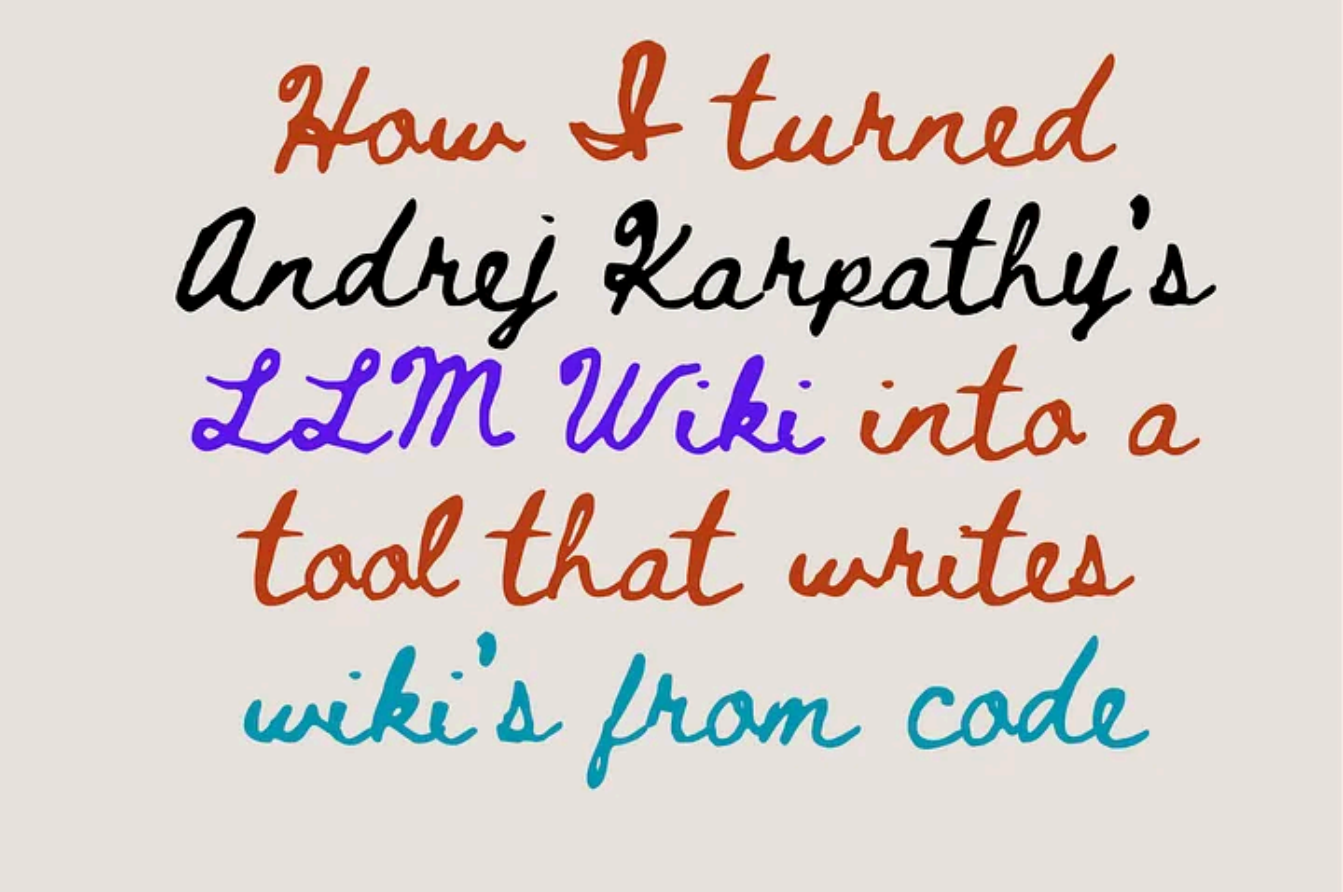
Listen Share ••• More

By Balu Kosuri

A couple of days ago I wrote an article about LLM Wiki, Andrej Karpathy’s pattern for turning piles of documents into a self-maintaining wiki. In the comments, someone asked whether the same idea could ingest code repositories instead of documents. That question stuck with me, and this project is what came out of it.

What it is: a small template that points the LLM Wiki pattern at a code repo and generates documentation from it: end-user docs, internal docs, whatever template you define.

How it works: you clone the repo, edit one config file, and run one install script. After that, every `git commit` triggers an agent in the background that reads the diff, updates the wiki, and keeps the docs in sync with the code.



My Repository: `https://github.com/balukosuri/docs-from-code-with-llm-wiki`
My Previous article: *I used Karpathy's LLM Wiki to build a knowledge base that maintains itself with AI*
Original idea by Andrej Kaarpathy: *Karpathy's llm-wiki.md*

The rest of this article is the story of why it works and where it breaks.

What this project is, and what it is not

I want to be blunt about this because it's the question that gets asked (and answered wrongly) most often.

This is a small personal project. It instantiates one of Andrej Karpathy's ideas (the `llm-wiki.md` pattern) and points it at a code repository instead of a document folder. Generating wiki's from code

The problem it addresses is specific: internal wikis and engineering knowledge bases die on the vine. They start well, drift out of date because maintenance is

nobody's job, and end up distrusted. A README that's 20% wrong is worse than no README, because now people are confidently misinformed.

This is not a replacement for a technical writer. I want to say that twice because it's important and because I am a technical writer. A writer brings judgment, narrative, audience empathy, editorial voice, and the willingness to push back on the engineers they work with. An LLM has none of those. What this tool produces is:

- a **draft** a writer can polish, not a finished doc
- an **internal reference** for developers, not a published product manual
- a **bookkeeper** for the 80% of content that decays as code changes, freeing human writers for the 20% that needs voice

If you have a writer on your team, this tool helps them by giving them a living, code-grounded reference to work from. It does not replace them. If you do not have a writer, this tool gives you something your team can trust for internal use, but do not mistake it for customer-ready documentation without a human in the loop.

Best fit: internal developer docs, CLI tools and libraries, solo devs and small teams without a docs function, teams that want a living reference alongside their human-authored product docs.

Poor fit without a human editor: shipped end-user product documentation, UI-heavy apps where behaviour depends on runtime data, compliance / legal / safety-critical content.

With that disclaimer out of the way, here's why I built it.

Why code documentation always goes out of date

Think about every README you've ever written. It started strong. Someone onboarded, skimmed it, and shipped a fix. A month later a feature got renamed. The README didn't. Three months later a whole subsystem was rewritten. The README didn't change. Six months later the README is a relic: wrong enough to mislead, right enough that nobody deletes it.

This is not a motivation problem. It is a **timing** problem.

Documentation decays because the moment the code changes is not the moment the docs get updated. By the time anyone notices, the developer who made the change has context-switched away and now lives in another world.

For my previous article this was easy to solve for static documents. You just "ingest" a PDF once and you're done. But code is not static. Every commit is a potential ingest. Nobody wants to type `ingest src/auth/login.ts` every time they save a file.

So I needed a different trigger.

Why git commit is the right trigger for updating documentation

Here is the move. The natural heartbeat of a code project is not "when the developer remembers to update docs". It is **`git commit`**. Every commit is an atomic unit of change: tested, reviewed (hopefully), and marked with a human-written subject. If the wiki updates itself on every commit, the docs never fall more than one commit behind the code.

That is what `docs-from-code-with-llm-wiki` does. A `post-commit` git hook fires in the background. It diffs `HEAD~1..HEAD`, hands the diff to whichever AI tool you configured, and the CLI updates the wiki in place. A few seconds later a follow-up commit lands with the subject `wiki: update (<sha>)`. Your `git commit` returned instantly. You kept coding.

Here is the whole flow:

```
git commit (code)
   │
   ▼
post-commit hook → nohup ingest.sh &   (returns immediately, you keep coding)
                           │
                           ├─ flock (prevents overlapping runs)
                           ├─ git diff <last-ingested>..HEAD
                           ├─ claude -p  (or cursor-agent, or codex)
                           │      └─ reads CLAUDE.md + config.yml
                           │         edits wiki/*.md in place
                           └─ git commit -m "wiki: update (<sha>)"
```

For a vibe coder, someone who codes in flow, commits when they feel like it, and does not want to be a documentation bureaucrat, this is the ideal loop. You never type “ingest”. You just commit. The docs appear.

What kind of documentation this AI wiki actually produces

The second question, after “what triggers it”, is “what does it produce”. And here I did not want to make one opinionated choice. Internal architecture docs, end-user README drafts, API reference, ADRs. Different projects want different mixes. Some want all of them; some want none.

So the template has **one configuration file** that flips categories on and off:

```
# .llmwiki/config.yml

cli: claude                # claude | cursor-agent | codex

doc_types:
  architecture: true       # Internal: module map, data flow
  api: true                # Public function / class / command
reference
  user: false              # End-user README drafts and usage pages
  decisions: true          # ADR-style decision records
  concepts: true           # Domain ideas discovered in the code

include:
  - "src/**"
  - "lib/**"

exclude:
  - "**/*.test.*"
  - "dist/**"

custom_types: []
#  - name: runbook
#    dir: runbooks/
#    trigger: "when the diff touches infra/ or ops/"
```

Turn on what you care about. Add your own categories under `custom_types`. That's the only knob.

On every ingest, the CLI re-reads this file. If you had `user: false` yesterday and flip it to `true` today, the next commit will start drafting end-user docs. No code changes required.

The enabled categories map one-to-one to folders under `wiki/`:

```
wiki/
  index.md          ← master catalog
  log.md            ← append-only activity log (great for `git blame` of docs)
  overview.md       ← big-picture synthesis
  glossary.md       ← every public identifier, flag, config key
  architecture/     ← per-subsystem pages (grouped by responsibility)
  api/              ← one page per exported function, class, or command
  user/             ← README drafts, how-tos, usage pages
  decisions/        ← ADR pages proposed from commit messages
  concepts/         ← domain ideas (e.g. "retry backoff", "job queue")
  sources/          ← one summary page per significant source file
```

Open it in Obsidian and the graph view colour-codes by folder. You can see at a glance which subsystems are hubs and which are orphans.

How to stop the LLM from hallucinating about your code

Code is an ungrateful source to summarize.

Code narrates *what* happens, like `function login(user, pass)`. It almost never narrates *why* the function exists, who calls it, what happens when it fails at 3am in production, or what the user sees when it does. An LLM summarizing a function without that context will happily fill in the gaps from its training distribution. If the code looks like other code it has seen, it will describe the behaviour the same way, even when the specific implementation does something unique.

How bad this is depends on how much of the behaviour actually lives in the code. Some code surfaces are readable end-to-end from the source: a function signature, a typed API schema, a config definition. Other code surfaces depend heavily on things the source does not narrate: styling, runtime data, user interaction,

environment. The more behaviour lives outside the code, the more the AI has to guess, and the more it will hallucinate.

There is no silver bullet for this, but two layered defences cover most of the damage:

1. Prompt discipline with a strict CLAUDE.md file

`CLAUDE.md` is the schema file that every AI CLI reads before it does anything. The version that ships with this template has seven hard rules. The key ones:

- **Cite or do not claim.** Every non-trivial statement must be followed by `(path:start-end)`. If you can't produce the citation, you are speculating. Stop and emit a `> TODO-VERIFY:` blockquote instead.
- **Never describe an API that is not at HEAD.** Don't write "this function accepts an optional `timeout`" if `timeout` is not in the current signature.
- **Never narrate runtime behaviour you cannot read.** No "this probably does X". If behaviour is only inferable from tests, cite the test file.
- **UI clamp.** For React components, templated HTML, and CSS-driven behaviour, do not describe what the user sees unless a test, snapshot, or storybook file confirms it.
- **Never cite anything outside this repo.** No external docs, no "frameworks like this usually...". Only code at HEAD.

Open in app ↗

Medium

2. A freshness check backed by real git SHAs

Every wiki page stores the `git hash-object` SHA of each file it cites, right there in the frontmatter:

```yaml
---
title: Login flow
type: module
sources:
  - path: src/auth/login.ts
    sha: 3f9a21bccd4e0f12abc34...
    lines: 1-120
```

```yaml
    - path: src/auth/session.ts
      sha: 0b12aa99cc8d77eeff01...
      lines: 15-88
  updated: 2026-04-17
  ---
```

The freshness script walks every page, re-runs `git hash-object` on each cited file, and reports any page whose citations no longer match. Two things make this cheap and reliable:

- Git blob SHAs are deterministic. Same content → same SHA. Always.
- You can run it in a pre-push hook, a CI job, or manually with `bash .llmwiki/freshness.sh`. It reads nothing but the working tree.

When a page is stale, you paste the stale list to your AI and say "re-generate these". The next ingest has real code in front of it, and the hallucinated claims go away.

Is this bulletproof? No. Nothing is. But it turns hallucinations from a silent, cumulative decay problem into a loud, dated, traceable one. You always know which pages might be wrong, and why.

How this differs from the original LLM Wiki for documents

Side-by-side comparison with the original technical-writer version I shipped two weeks ago:

Aspect	Documents version	Code version
Source of truth	`raw/` folder of PDFs, transcripts, clippings	The git repo at HEAD
Trigger	Human types "ingest"	`post-commit` hook fires automatically
Entity types	Source, Product, Feature, Persona, Concept, Style	Source, Module, API, User-doc, Decision, Concept
Citation format	"As noted in `[[onboarding-prd]]`"	`(src/auth/login.ts:45-62)` with a blob SHA in frontmatter
Update cadence	Whenever the human ingests	Every commit
Main failure mode	Stale when new documents arrive and aren't ingested	Hallucination from insufficient code context
Best fit	Research, writing, PM work	Any codebase, best on code whose public surface is readable from source

Same pattern, different dials.

What using this looks like in a real developer workflow

Imagine you're building a small project over the weekend. You don't want to stop and write docs. You just want to code.

You install the template once and forget about it. From that point on, every time you run `git commit`, a small agent runs in the background, reads what changed, and updates the wiki for you.

- You add a new function or endpoint → a page for it shows up in the wiki.
- You change a signature or config option → the relevant page updates itself.
- You rename or split a file → the wiki reshuffles itself to match.
- Something doesn't line up between the code and the docs → the wiki leaves a `TODO-VERIFY` note pointing at the mismatch, so you can catch it before shipping.

You never open the wiki while coding. You just commit. Later, when you want to share your project, write a blog post, or onboard someone, you open the `wiki/`

folder and most of the documentation is already there: overview, reference, glossary, and a graph of how the pieces connect.

The mental model is simple: **you write code, the agent writes docs, git is the handoff between the two.**

The honest limitations of AI generated code documentation

- **Hallucinations are mitigated, not eliminated.** The citation discipline in `CLAUDE.md` and the SHA-based freshness check reduce drift. They do not guarantee correctness. Any AI-authored claim can still be wrong.
- **UI-heavy code** is the weakest case. If you work on a React/Vue/Svelte app, turn `api` and `architecture` on, turn `user` off, and treat the output as a skeleton a human has to finish. Do not publish the generated end-user docs without review.
- **Large diffs** (massive refactors, rename-everything commits) get skipped by default above 2000 lines. You can lift the limit, but expect cost. The template prints a skip message rather than silently burning tokens.
- **Renames across files.** Git's rename detection is heuristic. If the AI sees "delete foo.ts + add bar.ts" it may not realise they are the same thing. The hook passes the commit subject through, so a subject like `refactor: rename foo.ts to bar.ts` gives it a strong hint.
- **Every commit costs money.** The hook calls an AI API on every matching commit. Tighten the globs and turn off unused doc types to keep the bill sane.
- **Not a single source of truth.** If you also have Confluence / Notion / a docs site, this sits alongside them. It documents the code-grounded slice; the rest stays where it is.
- **Private APIs vs public APIs.** The template defaults to documenting the public surface. If you want internals documented too, broaden the `include` glob and enable `architecture`.
- **Monorepos.** Works fine with one wiki per repo. For a monorepo, either run one wiki in the repo root (broad) or put a separate `.llmwiki/` and `wiki/` per package

(focused). The latter tends to be more useful.

- **CI-only workflows.** This template runs locally. Teams that want the wiki updated on merge-to-main rather than per-commit-per-developer should adapt the hook into a GitHub Action. The prompts and `CLAUDE.md` port straight over.
- **Windows.** Tested on macOS. Linux should work. Windows requires Git Bash or WSL; native cmd / PowerShell will not run the hook.
- **Quality tracks the CLI.** The wiki is only as good as the model you put behind it. Upgrade the model, upgrade the wiki.
- **Not a reviewer.** This documents code. It does not tell you the code is wrong, insecure, or could be simpler.

Common misconceptions about AI code documentation

A handful of misinterpretations worth clearing up in one place:

- **"This replaces technical writers."** It does not. It produces drafts and a living reference. The last mile (voice, narrative, audience empathy, editorial judgement) is still a human job.
- **"I can publish the generated output as end-user docs."** For internal use, often yes. For customer-facing content, always review. The hallucination risk on behaviour is real.
- **"It understands the whole codebase."** It understands the diff and the pages that cite the changed files. For a full sweep, ask your CLI directly to "re-read `wiki/index.md` and deep-scan". That's a different operation.
- **"No `CONTRADICTION` block means the wiki is correct."** Absence of flagged issues is not absence of issues. Run `freshness.sh` and spot-check the cited line ranges. Trust, but verify.
- **"The hook fixes bad code."** It documents whatever is there. It is not a reviewer or a refactor tool.
- **"One commit updates one wiki page."** One commit often touches 5–15 pages: glossary, index, overview, plus the affected entity pages. That is the design.

- **“Turning on every doc type gives me better docs.”** It gives you broader docs at higher cost. Narrow, enabled categories produce sharper pages.
- **“My team can use this instead of Confluence.”** Different tool, different purpose. This lives next to the code. Product specs, org knowledge, and customer-facing docs belong in their existing homes.
- **“A `wiki: update` commit that looks right means the wiki is right."** It means the AI thought it was right. Periodic human review is still required.

If you catch yourself thinking any of the left-hand versions, come back to this list.

How to install the LLM Wiki in your repo in a few minutes

```
git clone https://github.com/balukosuri/docs-from-code-with-llm-wiki.git my-prc
cd my-project
```

```
# Edit .llmwiki/config.yml: pick your CLI, toggle doc types
$EDITOR .llmwiki/config.yml

# Install the hook
bash .llmwiki/install-hook.sh

# Commit some code (even "initial commit" works)
git add . && git commit -m "start"

# Watch the wiki populate
tail -f .llmwiki/state/ingest.log
```

Open the folder in Obsidian as a vault. The overview page opens by default. Graph view is `Cmd+G`.

If you are vibe coding, the whole thing is: clone, edit one YAML file, run one script, then forget the wiki exists. Your commits write it for you.

Final thoughts

Documentation failures are almost never “we didn’t know how to write docs”. They are “the moment the code changed and the moment the docs should have changed are separated by too much time and too many context switches”.

Karpathy’s LLM Wiki pattern collapses that gap for documents by making an AI agent the wiki maintainer. This variant collapses it for code by making `git commit` the trigger and grounding every claim in `path:line` citations that a freshness check can verify.

The wiki is not a thing you maintain alone. It’s a thing the hook drafts and you refine. Your job shifts from bookkeeper to reviewer: write the code, commit it, and review what the hook proposes. The authoring that actually matters (voice, narrative, judgement, editorial taste) is still yours. The tool just removes the grunt work that used to stand between you and the parts of documentation that need a human.

If the previous article was about turning 15 scattered documents into a self-updating knowledge base, this one is about turning a live, evolving codebase into its own living documentation. The pattern is the same. The trigger is what changed.

My name is Balasubramanyam Kosuri, and I work as a technical writer. Connect with me on LinkedIn for more such content.

Repository: `https://github.com/balukosuri/docs-from-code-with-llm-wiki` *Previous article: I used Karpathy's LLM Wiki to build a knowledge base that maintains itself with AI Original idea: Karpathy's llm-wiki.md*

Andrej Karpathy | Documentation | AI | Code | Github

Follow

Written by Balu Kosuri

374 followers · 23 following

Just keep writing

No responses yet

Christian Broberg

What are your thoughts?